

Algorithmic Fairness

Anthony Kamau

1 Algorithmic Fairness

In this Exercise, we will measure the bias in a given dataset and apply some bias mitigation algorithms. You will need to install the AIF360 library using:

- `!pip install aif360`

This library contains implementation of a wide variety of bias mitigation algorithms including pre/in/post-processing algorithms. However, you will need to upload the dataset that you would like to work with in the AIF360 directory in Google colab.

To do so, download the dataset from the specified link on the GitHub. It is recommended to use the [German Credit Data](#). After downloading the data, go to Google colab and click on the button that points up as in the figure below. When you see the folders' tree, change to: `usr/local/lib/pythonX.XX/dist-packages/aif360/data/raw/german` and click on the three vertical dots next to that name then select Upload. Browse to the location where you saved the dataset and upload it.

First, we read the German dataset and replace the class label 2 by 0.

```
[44]: import numpy as np
import pandas as pd
from collections import Counter
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from matplotlib import pyplot as plt
from numpy import where
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import balanced_accuracy_score
from sklearn.metrics import f1_score
from sklearn.preprocessing import StandardScaler
```

```
[45]: # !pip install aif360
```

```
[46]: !mkdir -p /usr/local/lib/python3.13/dist-packages/aif360/data/raw/german
```

```
!cp /content/german.data /usr/local/lib/python3.13/dist-packages/aif360/data/
↳raw/german/
```

```
!ls -la /usr/local/lib/python3.13/dist-packages/aif360/data/raw/german/
```

```
total 92
drwxr-xr-x 2 root root 4096 Sep 27 22:28 .
drwxr-xr-x 7 root root 4096 Sep 27 21:49 ..
-rw-r--r-- 1 root root 79793 Sep 27 22:44 german.data
-rw-r--r-- 1 root root 685 Sep 27 21:49 README.md
```

```
[47]: from aif360.metrics import BinaryLabelDatasetMetric
      from aif360.datasets import GermanDataset
      from aif360.algorithms.preprocessing.lfr import LFR

      german_data = GermanDataset()
      german_df = german_data.convert_to_dataframe()
      german_df = german_df[0]
      german_df["credit"].replace({2.0: 0}, inplace=True)
      german_df
```

/tmp/ipykernel_1531/419341191.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
german_df["credit"].replace({2.0: 0}, inplace=True)
```

```
[47]:
```

	month	credit_amount	investment_as_income_percentage	residence_since	\
0	6.0	1169.0	4.0	4.0	
1	48.0	5951.0	2.0	2.0	
2	12.0	2096.0	2.0	3.0	
3	42.0	7882.0	2.0	4.0	
4	24.0	4870.0	3.0	4.0	
..	...	...	...	...	
995	12.0	1736.0	3.0	4.0	
996	30.0	3857.0	4.0	4.0	
997	12.0	804.0	4.0	4.0	
998	45.0	1845.0	4.0	4.0	
999	45.0	4576.0	3.0	4.0	

	age	number_of_credits	people_liable_for	sex	status=A11	status=A12	\
0	1.0	2.0	1.0	1.0	1.0	0.0	
1	0.0	1.0	1.0	0.0	0.0	1.0	
2	1.0	1.0	2.0	1.0	0.0	0.0	
3	1.0	1.0	2.0	1.0	1.0	0.0	
4	1.0	2.0	2.0	1.0	1.0	0.0	
..	...	...	...	...	...	...	
995	1.0	1.0	1.0	0.0	0.0	0.0	
996	1.0	1.0	1.0	1.0	1.0	0.0	
997	1.0	1.0	1.0	1.0	0.0	0.0	
998	0.0	1.0	1.0	1.0	1.0	0.0	
999	1.0	1.0	1.0	1.0	0.0	1.0	

	housing=A153	skill_level=A171	skill_level=A172	skill_level=A173	\
0	...	0.0	0.0	0.0	1.0
1	...	0.0	0.0	0.0	1.0
2	...	0.0	0.0	1.0	0.0
3	...	1.0	0.0	0.0	1.0
4	...	1.0	0.0	0.0	1.0
..	...	...	...	...	...
995	...	0.0	0.0	1.0	0.0
996	...	0.0	0.0	0.0	0.0
997	...	0.0	0.0	0.0	1.0
998	...	1.0	0.0	0.0	1.0
999	...	0.0	0.0	0.0	1.0

	skill_level=A174	telephone=A191	telephone=A192	foreign_worker=A201	\
0	0.0	0.0	1.0	1.0	
1	0.0	1.0	0.0	1.0	
2	0.0	1.0	0.0	1.0	
3	0.0	1.0	0.0	1.0	
4	0.0	1.0	0.0	1.0	
..	...	...	...	...	
995	0.0	1.0	0.0	1.0	
996	1.0	0.0	1.0	1.0	
997	0.0	1.0	0.0	1.0	
998	0.0	0.0	1.0	1.0	
999	0.0	1.0	0.0	1.0	

	foreign_worker=A202	credit
0	0.0	1.0
1	0.0	0.0
2	0.0	1.0
3	0.0	1.0
4	0.0	0.0
..	...	...
995	0.0	1.0

996	0.0	1.0
997	0.0	1.0
998	0.0	0.0
999	0.0	1.0

[1000 rows x 59 columns]

Check the example notebooks at [aif360/examples](#) to learn how to specify the privileged and unprivileged groups according to the sensitive attributes.

```
[48]: # Define the privileged and unprivileged
privileged_groups = [{'sex': 1, 'age': 1}]
unprivileged_groups = [{'sex': 0, 'age': 0}]

# privileged_groups = [{'sex': 1}]
# unprivileged_groups = [{'sex': 0}]
```

Check if the data is biased or not using the measures that can be computed from the dataset directly (do not require the outcome of the classifier).

```
[49]: metric_orig = BinaryLabelDatasetMetric(german_data,
                                             ↪unprivileged_groups=unprivileged_groups,
                                             ↪privileged_groups=privileged_groups)
print("Disparate impact (of original labels) = %f" % metric_orig.
      ↪disparate_impact())
print("Difference in statistical parity (of original labels) = %f"
      ↪metric_orig.statistical_parity_difference())
print("Individual fairness metric (consistency) = %f" % metric_orig.
      ↪consistency())
```

Disparate impact (of original labels) = 0.747630

Difference in statistical parity (of original labels) = -0.186462

Individual fairness metric (consistency) = 0.681600

/tmp/ipykernel_1531/4243434661.py:7: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)

```
print("Individual fairness metric (consistency) = %f" %
metric_orig.consistency())
```

Apply one of the bias mitigation algorithms and display the values of the fairness measure that you can compute.

```
[50]: # Train-test split WITH stratification
german_df_copy = german_df.copy()
german_df_copy.drop(['sex', 'age'], axis = 1)
```

```

X_ger = german_df_copy.loc[:, german_df_copy.columns != 'credit']
y_ger = german_df_copy.loc[:, german_df_copy.columns == 'credit'].values
X_train_g, X_test_g, y_train_g, y_test_g = train_test_split(X_ger, y_ger,
↳test_size=0.30,

                                                                    shuffle = True,
                                                                    stratify = y_ger)

```

```

[51]: # Train a logistic regression model
      # Train the model with the training subset
      logreg = LogisticRegression()
      logreg.fit(X_train_g, y_train_g)

      # Test the model on the test subset
      result_LR = logreg.predict(X_test_g)
      # Display the confusion matrix to check the performance of your model
      confusion_matrix(result_LR, y_test_g)

```

/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:1408:
DataConversionWarning: A column-vector y was passed when a 1d array was
expected. Please change the shape of y to (n_samples,), for example using
ravel().

```

y = column_or_1d(y, warn=True)

```

/usr/local/lib/python3.13/dist-packages/sklearn/linear_model/_logistic.py:465:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```

n_iter_i = _check_optimize_result(

```

```

[51]: array([[ 40,  35],
           [ 50, 175]])

```

```

[52]: # Baseline Method
      dataset_orig_train, dataset_orig_test = german_data.split([0.7], shuffle=True)

      #1) Transforming Adult Dataset

      #Required Inputs:
      # Input reconstruction quality - Ax
      # Fairness constraint - Az
      # Output prediction error - Ay

      #Scaling the dataset

```

```

scale_orig = StandardScaler()
#scaled dataset together with its labels is needed
dataset_orig_train.features = scale_orig.fit_transform(dataset_orig_train.
    ↪features)
dataset_orig_test.features = scale_orig.transform(dataset_orig_test.features)

#LFR itself contains logistic regression sinc it uses signoid functions
LFR = LFR(unprivileged_groups=unprivileged_groups,
    privileged_groups=privileged_groups,
    k=5, Ax=0.1, Ay=1.0, Az=100.0, verbose=1)

TR = LFR.fit(dataset_orig_train, maxiter=5000, maxfun=5000)

# Transform training data and align features
dataset_transf_train = TR.transform(dataset_orig_train)

```

```

step: 0, loss: 1.7761295123924417, L_x: 2.433640511644765, L_y:
0.7026690146337744, L_z: 0.008300964465941908
step: 250, loss: 1.7761296200933154, L_x: 2.4336406119258696, L_y:
0.7026689722626709, L_z: 0.008300965866380577
step: 500, loss: 2.21707735172887, L_x: 2.42460226452118, L_y:
0.2092196727788271, L_z: 0.01765397452497925
step: 750, loss: 1.2017423574588477, L_x: 2.431288440136097, L_y:
0.523149650885523, L_z: 0.004354638625597151
step: 1000, loss: 0.9042246336416836, L_x: 2.431725944898342, L_y:
0.3643816548513618, L_z: 0.0029667038430048753
step: 1250, loss: 1.0389182091016538, L_x: 2.4345490344716954, L_y:
-0.1206763668980087, L_z: 0.009161396725524929
step: 1500, loss: 0.69298310814971, L_x: 2.4328754538594373, L_y:
0.2044674566997324, L_z: 0.002452281060640338
step: 1750, loss: 0.6929853212214756, L_x: 2.4328755342618438, L_y:
0.20446741188160802, L_z: 0.0024523035591368313
step: 2000, loss: 0.5802267127352863, L_x: 2.433580320511203, L_y:
0.036537582932260775, L_z: 0.0030033109775190526
step: 2250, loss: 0.22248475049777497, L_x: 2.434021857414605, L_y:
-0.485569238273362, L_z: 0.004646518030296765
step: 2500, loss: 0.06364959262180037, L_x: 2.434002336136336, L_y:
-0.5574491392593118, L_z: 0.003776984982674786
step: 2750, loss: -0.23717252237018693, L_x: 2.434119939774294, L_y:
-0.8584625102685212, L_z: 0.003778779939209048
step: 3000, loss: -0.5938682676975163, L_x: 2.434149543581087, L_y:
-1.1673013331297635, L_z: 0.003300181110741385
step: 3250, loss: -0.5938685581560077, L_x: 2.434149563454919, L_y:
-1.167301375955085, L_z: 0.0033001786145358536
step: 3500, loss: -10.102726709455391, L_x: 2.4341386585774263, L_y:
-10.607132283083047, L_z: 0.0026099170776991256
step: 3750, loss: -9.832027557346738, L_x: 2.4219178023824686, L_y:
-10.607132283083047, L_z: 0.005329129454980608

```

```

step: 4000, loss: -10.184031222986343, L_x: 2.4302972374034315, L_y:
-10.607132283083047, L_z: 0.0018007133635636163
step: 4250, loss: -10.218663135948319, L_x: 2.4297867082576463, L_y:
-10.607132283083047, L_z: 0.0014549047630896372
step: 4500, loss: -10.109862595839086, L_x: 2.427668367753153, L_y:
-10.607132283083047, L_z: 0.0025450285046864527
step: 4750, loss: -10.30969098198896, L_x: 2.4291635124901134, L_y:
-10.607132283083047, L_z: 0.0005452494984507672
step: 5000, loss: -10.309690535383881, L_x: 2.4291635520222066, L_y:
-10.607132283083047, L_z: 0.0005452539249694522

```

```

[59]: #check if the labels are transformed: counts the num. of transformed labels
k=0
for i in range(len(dataset_orig_train.labels)):
    if(dataset_transf_train.labels[i] == dataset_orig_train.labels[i]):
        pass
    else:
        k+=1

k

```

[59]: 206

```

[60]: #Fairness Performance of Datasets Before Classification

#Constructing two functions to call the desired metrics
metric_transf_train = BinaryLabelDatasetMetric(dataset_transf_train,
                                                unprivileged_groups =_
↳unprivileged_groups,
                                                privileged_groups =_
↳privileged_groups)

print("Disparate impact ratio (of transformed labels) = %f"
      % metric_transf_train.disparate_impact())
print("Difference in statistical parity (of transformed labels) = %f"
      % metric_transf_train.statistical_parity_difference())
print("Individual fairness metric 'consistency' = %f" % metric_transf_train.
↳consistency())

```

Disparate impact ratio (of transformed labels) = 1.000000

Difference in statistical parity (of transformed labels) = 0.000000

Individual fairness metric 'consistency' = 1.000000

/tmp/ipykernel_1531/89499636.py:12: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)

```

    print("Individual fairness metric 'consistency' = %f" %
metric_transf_train.consistency())

```

```
[55]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

```
[56]: %%capture
#!apt-get update -qq
!apt-get install -y texlive-xetex texlive-fonts-recommended texlive-latex-extra
```

```
[57]: %%capture
!cp "/content/drive/MyDrive/Colab Notebooks/algorithmic_fairness.ipynb" ./

!jupyter nbconvert --to latex "algorithmic_fairness.ipynb"

with open("algorithmic_fairness.tex", "r", encoding="utf-8") as f:
    tex = f.read()

title_block = (
    r"\title{Algorithmic Fairness}" + "\n" +
    r"\author{Anthony Kamau}" + "\n" +
    r"\date{}" + "\n"
)

tex = tex.replace(r"\begin{document}", title_block + r"\begin{document}")

if r"\maketitle" not in tex:
    tex = tex.replace(r"\begin{document}", r"\begin{document}" + "\n" +
        r"\maketitle", 1)

with open("algorithmic_fairness.tex", "w", encoding="utf-8") as f:
    f.write(tex)

!xelatex algorithmic_fairness.tex
!xelatex algorithmic_fairness.tex
```

```
[58]: from google.colab import files
files.download("algorithmic_fairness.pdf")
```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>